

INE Product Price Tracker

Complete Implementation Plan & Technical Handoff

Recommended approach: React + Express + Supabase + Playwright + external cron.

1. Assignment Goal

Build a full-stack application where users search the INE mock storefront, select products to track, scrape price and stock every 2 hours, view historical data, and inspect every scrape attempt. The assignment identifies scraper reliability as the core challenge.

2. Required Stack

Layer	Technology	Purpose
Frontend	React.js / Vercel	Search, tracking dashboard, history, logs
Backend	Node.js + Express / Render	REST API and scraper orchestration
Database	Supabase PostgreSQL	Tracked products, price history, scrape logs
Scraper	Playwright	Browser-based scraping where JS is genuinely required
Scheduler	cron-job.org	Trigger every 2 hours

3. Scraping Decision

Use Playwright as the primary scraper. Investigation of the storefront showed that obtaining the current price involves browser-side challenge/attestation behavior, WASM execution, proof-of-work, a session token, and encrypted price data. Reproducing all of this manually is possible but unnecessarily complex for the assignment. A real Chromium browser lets the storefront's own JavaScript execute its intended flow.

4. Full Protocol Found During Investigation

API-level flow: GET /api/challenge → build browser attestation → SHA-256 fingerprint hash → derive WASM seed → execute challenge WASM → solve PoW nonce → calculate derive → POST /api/session → receive Bearer token → GET /api/products/{productId}/price → decrypt response → obtain price/stock.

5. Playwright Flow

Launch Chromium → open product page → perform required interaction (for example, hover over the price area if required) → wait for dynamic content → extract price and stock → validate → return result → store history/log.

6. Database Schema

tracked_products: id, product_id, product_name, product_url, tracking_enabled, created_at, updated_at.

price_history: id, tracked_product_id, price, stock, currency, scraped_at.

scrape_logs: id, tracked_product_id, started_at, completed_at, status, attempt, price, stock, error_message, duration_ms.

7. Backend API

GET /api/products/search?q=

GET /api/tracked-products

POST /api/tracked-products

GET /api/tracked-products/:id/history

GET /api/tracked-products/:id/logs

POST /api/scrape/:productId
POST /api/scrape/run

8. Scraper Implementation

Create a scraper service with browser/context setup; product navigation; required interaction; explicit waits; stable selectors; price/stock extraction; validation; bounded retries; error handling; scrape logging; and HEADLESS=true/false support.

9. Reliability Rules

Never save an empty or invalid price/stock. A failed scrape must not overwrite the last known valid history. Record every attempt honestly. Use bounded retry/backoff. Use explicit navigation, selector, and overall scrape timeouts. Only insert price_history after validation succeeds.

10. Retry Model

Attempt 1 → failure → backoff → attempt 2 → failure → backoff → attempt 3 → final failure. On final failure, write FAILED to scrape_logs and do not insert a new price_history row.

11. Frontend

Keep the UI simple: product search, Track button, tracked-products dashboard, current/last-known price and stock, price/stock history table or chart, scrape logs, and a manual Scrape Now button.

12. Scheduling

Do not rely on setInterval in Render. Configure cron-job.org to call POST /api/scrape/run every 2 hours. The endpoint loads tracked products, scrapes them, validates results, and writes history/logs.

13. Headed Demonstration

Run with HEADLESS=false. Record 2–4 minutes showing the browser, scraping behavior, and handling of a slow or failing response. This directly addresses the assignment's observable-run requirement.

14. Project Structure

```
frontend/  
src/components/  
src/pages/  
src/services/api.js  
src/App.jsx  
  
backend/  
src/routes/  
src/controllers/  
src/services/  
src/scraper/browser.js  
src/scraper/productScraper.js  
src/scraper/retry.js  
src/scraper/scraperService.js  
src/db/  
src/server.js
```

15. Implementation Order

Phase 1: Prove Playwright can scrape ONE product's price and stock.

Phase 2: Add waits, timeouts, retries, validation, errors, logging, headed/headless mode.

Phase 3: Add Supabase tables and persistence.

Phase 4: Build Express APIs.

Phase 5: Build React dashboard.

Phase 6: Configure 2-hour cron.

Phase 7: Deploy to Vercel/Render/Supabase.

Phase 8: Reliability testing, recording, README, design note, submission.

16. Testing Checklist

- Search partial/full product names
- Track a product
- Scrape price
- Scrape stock
- Handle delayed content
- Handle slow responses
- Retry transient failures
- Record final failures
- Never store invalid data
- Preserve last valid history after failure
- Show per-attempt logs
- Manual scrape works
- Scheduled scrape works
- Headed mode works
- Production deployment works

17. Submission Checklist

- Live hosted site
- Public GitHub repository
- 2–4 minute headed recording
- README with setup, schedule, environment variables
- Design note explaining reliability, trade-offs, and AI-tool mistakes/corrections
- PDF resume

18. Technical Findings to Preserve for Debugging

The storefront's JS flow was decoded enough to establish: `/api/challenge` supplies challenge data; `att` is a JSON browser/environment + interaction snapshot; its SHA-256 is used in `seed/derive`; challenge WASM is instantiated and its exported `f` is called; PoW searches for a nonce whose SHA-256(salt:nonce) starts with the required zero count; `/api/session` receives challenge fields plus nonce, `derive`, `wasmOut`, `att`, and `productId`; `/api/products/{id}/price` requires a Bearer token; and the encrypted response is decrypted client-side using a SHA-256-derived key and repeating-key XOR. These details are primarily fallback/debugging context—the implementation should first use Playwright.

19. Key Principle

Do not build the entire application first. First prove the scraper works reliably for one product. Then add reliability. Then persistence, API, UI, scheduling, and deployment. The scraper is the highest-priority component because reliability and correctness are the central evaluation criteria.

20. One-Sentence Summary

A React + Express + Supabase price-tracking application whose Playwright scraper reliably retrieves price and stock from INE's deliberately difficult mock storefront every two hours, with retries, validation, historical storage, and honest scrape logging.

Source Basis

This plan is based on the uploaded INE Software Engineer Intern Assignment PDF and the storefront JavaScript investigation from the preceding conversation. The assignment states that reliability is the core of the assessment, requires 2-hour scraping, honest logging, an observable headed run, and the specified Vercel/Render/Supabase/free-tier deployment approach.